



MINISTÉRIO DE ENSINO SUPERIOR, CIÊNCIA, TECNOLOGIA E INOVAÇÃO
UNIVERSIDADE METODISTA DE ANGOLA
Coordenação de Mestrado em Engenharia Informática

JI SYNC

Sincronização Inteligente de Documentos

Manual de Utilização

Autores:

1. Ilda Luzia da Costa Pedro
2. Massonguele João

Índice

Índice.....	1
1. Introdução	5
1.1. Objetivo do projeto.....	5
1.2. Principais funcionalidades.....	5
2. Arquitetura do Sistema	6
2.1. Componentes principais	6
2.2. Sincronização em tempo real: o percurso de uma edição	6
Etapa 1 - No cliente que escreve	6
Etapa 2 - No servidor.....	6
Etapa 3 - Nos outros clientes (reconciliação)	7
Etapa 4 - Persistência duradoura (em paralelo)	7
Modelo de consistência	7
2.3. Estratégia de autosave (Redis → PostgreSQL).....	7
2.4. Autenticação e sessões	7
2.5. Controle de acesso e isolamento de dados	8
3. Primeiros Passos	9
3.1. Criar conta	9
3.2. Iniciar sessão	9
3.3. Recuperar palavra-passe.....	9
4. Dashboard - Os Meus Documentos	10
4.1. Criar um novo documento.....	10
4.2. Vistas: grelha e lista	10
4.3. Ações rápidas em cada documento	10
4.4. Indicadores em tempo real	10
5. Projetos e Pastas.....	11
5.1. Criar um projeto	11
5.2. Pastas.....	11

5.3. Membros do projeto	11
6. Editor de Texto (Word).....	12
6.1. Formatação	12
6.2. Quebra de página.....	12
6.3. Colaboração em tempo real.....	12
6.4. Identificação do autor.....	12
6.5. Comentários	12
6.6. Histórico de versões	12
6.7. Exportação.....	13
6.8. Partilha	13
6.9. Controlos de janela.....	13
7. Folha de Cálculo (Excel)	14
7.1. Formatação	14
7.2. Múltiplas folhas.....	14
7.3. Linhas, colunas e congelamento.....	14
7.4. Colaboração em tempo real.....	14
7.5. Exportação.....	14
7.6. Partilha	14
8. Colaboração e Permissões.....	15
8.1. Limites de sala.....	15
8.2. Papéis de acesso a um documento.....	15
9. Segurança.....	16
10. Limitações e Trabalho Futuro	17
10.1. O modelo de sincronização atual	17
10.2. Alternativa 1 - Operational Transformation (OT).....	17
10.3. Alternativa 2 - CRDT (Conflict-free Replicated Data Type).....	18
10.4. Porque não foi implementado nesta versão.....	18
11. Conclusão.....	19
11.1. Tecnologias utilizadas	19

1. Introdução

O JI Sync é uma plataforma web de colaboração em tempo real para documentos de texto (Word) e folhas de cálculo (Excel), inspirada em ferramentas como o Google Docs e o Microsoft Office 365. Permite que várias pessoas editem o mesmo documento em simultâneo, veem quem está a editar, comentam trechos de texto, e organizam o trabalho em projetos e pastas.

Este manual foi desenvolvido no âmbito da unidade curricular de Sistemas Distribuídos, servindo como documentação de apoio à apresentação do projeto.

1.1. Objetivo do projeto

Demonstrar, na prática, conceitos fundamentais de sistemas distribuídos: comunicação em tempo real entre múltiplos clientes (WebSockets), consistência e sincronização de estado partilhado, autenticação e controlo de acesso, e estratégias de persistência e cache para reduzir a carga sobre a base de dados.

1.2. Principais funcionalidades

- Edição colaborativa em tempo real de documentos Word e Excel
- Organização por projetos, pastas e documentos
- Comentários e identificação do autor de cada trecho de texto
- Histórico de versões, com possibilidade de restaurar
- Partilha de documentos com permissões (editor / leitor)
- Exportação para .docx, .xlsx, .csv e .pdf
- Notificações em tempo real de atividade nos documentos

2. Arquitetura do Sistema

A aplicação segue uma arquitetura cliente-servidor com comunicação em tempo real, combinando pedidos REST tradicionais com uma camada de WebSockets para a colaboração ao vivo.

2.1. Componentes principais

Componente	Tecnologia	Função
Frontend	HTML, CSS, JavaScript	Interface do utilizador - dashboard, editor de texto (Quill.js) e folha de cálculo (Handsontable)
Backend	Node.js + Express	API REST (autenticação, documentos, projetos, comentários, partilha)
Tempo real	Socket.IO	Sincronização de edições, cursores, presença e notificações entre clientes
Base de dados	PostgreSQL	Persistência definitiva de utilizadores, documentos, versões e comentários
Cache	Redis	Absorve as escritas em tempo real antes de estas serem gravadas em lote na base de dados

2.2. Sincronização em tempo real: o percurso de uma edição

Quando um utilizador escreve uma letra no editor de texto, a atualização segue um percurso preciso, com quatro etapas:

Etapa 1 - No cliente que escreve

O Quill.js dispara um evento text-change. O sistema espera 500 milissegundos (debounce) - se a pessoa continuar a escrever, o temporizador reinicia. Só quando há uma pausa é que o HTML completo do documento é enviado ao servidor via Socket.IO (evento word-change).

Etapa 2 - No servidor

O servidor não grava de imediato no PostgreSQL. Guarda o novo estado no Redis (memória rápida) e reencaminha-o imediatamente a todos os outros clientes ligados à mesma sala (evento word-update).

Etapa 3 - Nos outros clientes (reconciliação)

Em vez de substituir todo o conteúdo local pelo que chegou, o que faria o cursor saltar e podia apagar texto que a pessoa estivesse a escrever nesse instante, o sistema calcula a diferença (Delta.diff() do Quill) entre o conteúdo atual e o recebido, e aplica só essa diferença. É esta técnica que permite ao cursor de cada pessoa manter-se estável durante a colaboração.

Etapa 4 - Persistência duradoura (em paralelo)

A cada 10 segundos, um processo de fundo (autosaveService) verifica que salas tiveram alterações pendentes, lê o estado mais recente do Redis, e só então grava no PostgreSQL, criando uma nova versão no histórico quando o conteúdo mudou de facto.

Modelo de consistência

Esta arquitetura garante consistência eventual: todos os clientes convergem para o mesmo estado, mas não instantaneamente nem com garantias formais de ordenação. A resolução de conflitos é do tipo "last-writer-wins" à granularidade da janela de debounce (~500 ms). Isto quer dizer, se duas pessoas editarem partes diferentes do documento, a reconciliação por diff normalmente resulta bem; se editarem exatamente o mesmo trecho dentro da mesma janela de meio segundo, existe a possibilidade teórica de uma edição ser silenciosamente sobreposta pela outra. Esta limitação, e as alternativas mais robustas (Operational Transformation e CRDT), são discutidas em detalhe na secção 10, Limitações e Trabalho Futuro.

2.3. Estratégia de autosave (Redis → PostgreSQL)

Gravar na base de dados a cada tecla digitada por cada utilizador seria impraticável a nível de desempenho. Em vez disso, o sistema segue uma estratégia em duas camadas:

- Cada alteração é imediatamente guardada no Redis e transmitida aos outros colaboradores via Socket.IO, latência mínima.
- Um processo de fundo (autosaveService) percorre, a cada 10 segundos, apenas as salas com alterações pendentes, e grava o estado consolidado no PostgreSQL, criando também uma versão no histórico quando o conteúdo muda.

Esta separação entre "escrita rápida e frequente" (Redis) e "escrita lenta e duradoura" (PostgreSQL) é um padrão comum em sistemas distribuídos para equilibrar desempenho e durabilidade.

2.4. Autenticação e sessões

A autenticação usa JSON Web Tokens (JWT): no login, o servidor gera um token assinado contendo o identificador do utilizador, que é enviado em todos os pedidos seguintes (REST e WebSocket)

através do cabeçalho Authorization. O servidor nunca precisa de consultar uma tabela de sessões — a validade do pedido é verificada só com a assinatura do token.

2.5. Controlo de acesso e isolamento de dados

Cada documento pertence a um dono (quem o criou) e, opcionalmente, a um projeto. O acesso é verificado em três camadas independentes, para garantir que a regra é sempre respeitada, seja qual for o caminho usado para tentar aceder:

- Nas rotas REST (abrir, editar, eliminar, histórico)
- Na entrada da sala de edição em tempo real (WebSocket)
- Na listagem do dashboard (só mostra o que o utilizador criou ou com que foi partilhado)

3. Primeiros Passos

3.1. Criar conta

Na página inicial, seleciona "Cadastre-se agora". É pedido nome de utilizador, email, palavra-passe e, opcionalmente, organização e função. Após o registo, é redirecionado para o login.

3.2. Iniciar sessão

Introduz o nome de utilizador e a palavra-passe. Por segurança, após 5 tentativas falhadas consecutivas, a conta fica temporariamente bloqueada durante 15 minutos.

3.3. Recuperar palavra-passe

A opção "Esqueceu a senha?" no ecrã de login envia um token de recuperação válido por 15 minutos.

4. Dashboard - Os Meus Documentos

É o ecrã principal após o login. Mostra todos os documentos criados pelo utilizador, mais os que foram partilhados com ele, nunca documentos de outras pessoas sem autorização.

4.1. Criar um novo documento

O botão "Novo documento" abre um formulário onde se escolhe o tipo (Word ou Excel) e o nome. O documento é criado já com um modelo em branco.

4.2. Vistas: grelha e lista

É possível alternar entre uma vista em grelha (cartões com pré-visualização) e uma vista em lista (mais compacta, útil quando há muitos documentos).

4.3. Ações rápidas em cada documento

- Renomear : abre uma janela para escrever o novo nome
- Eliminar : pede confirmação antes de remover definitivamente
- Histórico : mostra todas as versões guardadas, com autor e hora, e permite restaurar qualquer uma delas

4.4. Indicadores em tempo real

Cada documento mostra um indicador verde quando alguém está a editá-lo naquele momento, e o dashboard recebe notificações instantâneas ("Fulano está a editar X") sem ser preciso atualizar a página.

5. Projetos e Pastas

Para organizar documentos por equipa, cliente ou departamento, o JI Sync oferece uma estrutura de Projetos, cada um podendo conter Pastas (um nível), e dentro delas, Documentos.

5.1. Criar um projeto

Na página "Projetos", o botão "Novo projeto" pede um nome e, opcionalmente, uma descrição. Quem cria o projeto torna-se automaticamente o seu "dono".

5.2. Pastas

Dentro de um projeto, o botão "Nova pasta" cria uma pasta na raiz do projeto. Os documentos podem ser criados diretamente na raiz ou dentro de uma pasta específica.

5.3. Membros do projeto

O ícone de membros permite adicionar pessoas ao projeto pelo email. Todos os membros de um projeto têm acesso total a todos os seus documentos e pastas — não há distinção de papéis dentro de um projeto (ao contrário da partilha de documentos individuais, explicada na secção 8).

6. Editor de Texto (Word)

O editor de texto usa a biblioteca Quill.js, oferecendo uma experiência semelhante ao Microsoft Word, com colaboração em tempo real.

6.1. Formatação

- Títulos (Título 1, 2, 3) e texto normal
- Tipo e tamanho de letra
- Negrito, itálico, sublinhado, rasurado, sub/superescrito
- Cor do texto e cor de fundo
- Listas numeradas e com marcadores, indentação
- Alinhamento do texto
- Citação em bloco, bloco de código, hiperligações e imagens

6.2. Quebra de página

O botão de quebra de página insere um separador visual que força uma nova página, tanto na visualização como nos ficheiros exportados (Word e PDF), útil para organizar documentos longos em secções.

6.3. Colaboração em tempo real

Vários utilizadores podem editar o mesmo documento em simultâneo. Cada pessoa tem uma cor atribuída, visível no cursor dos outros colaboradores e na lista de "Utilizadores online" na barra lateral.

6.4. Identificação do autor

Ao passar o rato sobre qualquer trecho do documento, aparece uma dica com o nome de quem escreveu aquele texto, útil em documentos com várias contribuições.

6.5. Comentários

Seleciona um trecho de texto e clica no ícone de comentário para adicionar uma anotação. O trecho fica realçado; o comentário aparece na barra lateral, com autor e hora, podendo ser resolvido (esbate o realce, mantém o histórico) ou eliminado.

6.6. Histórico de versões

A barra lateral lista as versões guardadas automaticamente (a cada alteração relevante) e permite restaurar qualquer uma delas.

6.7. Exportação

O botão "Exportar" permite descarregar o documento em formato Word (.docx) ou PDF (.pdf), preservando a formatação e as quebras de página.

6.8. Partilha

O botão "Partilhar" permite convidar outra pessoa pelo email, escolhendo se pode editar ou só visualizar o documento. A lista de pessoas com acesso pode ser consultada e alterada a qualquer momento.

6.9. Controlos de janela

No canto superior direito, os botões de minimizar, maximizar (ecrã inteiro) e fechar permitem sair do documento de forma clara, sem depender da seta de "voltar" do navegador.

7. Folha de Cálculo (Excel)

A folha de cálculo usa a biblioteca Handsontable, com uma experiência semelhante ao Microsoft Excel.

7.1. Formatação

- Negrito, itálico, rasurado
- Alinhamento horizontal
- Cor de preenchimento e cor do texto
- Tamanho de letra
- Mesclar / desmesclar células
- Contornos (aplicar / remover)
- Formato numérico (Geral, Número, Moeda, Percentagem)

7.2. Múltiplas folhas

É possível criar, mudar entre, renomear (duplo-clique no separador) e eliminar várias folhas dentro do mesmo documento, tal como no Excel. Todas as folhas são guardadas e sincronizadas em tempo real entre colaboradores.

7.3. Linhas, colunas e congelamento

Inserir/remover linhas e colunas, e congelar a linha de cabeçalho para facilitar a navegação em folhas grandes.

7.4. Colaboração em tempo real

Tal como no editor de texto, mostra quem está online, com cores distintas, e respeita os mesmos limites e papéis de acesso.

7.5. Exportação

Exporta para Excel (.xlsx, incluindo todas as folhas), CSV (a folha atual) ou PDF (tabela formatada).

7.6. Partilha

Funciona exatamente como no editor de texto (secção 6.8).

8. Colaboração e Permissões

8.1. Limites de sala

Nº de pessoas na sala	Comportamento
0 a 20	Todos entram em modo de edição completa
21 a 50	Novas entradas ficam em modo só-leitura
Mais de 50	Documento bloqueado - não são aceites mais entradas

8.2. Papéis de acesso a um documento

- **Dono** : quem criou o documento; único que pode eliminá-lo ou geri-lo a partilha
- **Editor** : convidado com permissão de edição; acesso igual ao dono, exceto eliminar
- **Leitor** : convidado só de visualização; entra sempre em modo de leitura, independentemente da capacidade da sala

Estas regras são aplicadas tanto nos pedidos REST como na sessão de edição em tempo real, para que não haja forma de as contornar.

9. Segurança

- Palavras-passe guardadas com hash bcrypt (nunca em texto simples)
- Autenticação por JWT em todas as rotas protegidas, REST e WebSocket
- Bloqueio temporário da conta após tentativas de login falhadas repetidas
- Isolamento de dados: um utilizador só acede a documentos que criou, a que foi convidado, ou que pertencem a um projeto do qual é membro
- Verificação de acesso aplicada em três camadas independentes (REST, WebSocket e listagem), para consistência

10. Limitações e Trabalho Futuro

Esta secção descreve, com honestidade técnica, o modelo de sincronização atual, as suas limitações conhecidas, e o que seria necessário para o levar mais longe, informação especialmente relevante para uma discussão académica sobre sistemas distribuídos.

10.1. O modelo de sincronização atual

O JI Sync usa uma técnica que se pode descrever como "snapshot + reconciliação por diff" (detalhada na secção 2.2): cada alteração envia o documento completo (ou a folha de cálculo completa) após uma pausa na escrita, e cada cliente que a recebe calcula e aplica só a diferença face ao que já tinha. Não é Operational Transformation (OT) nem um Conflict-free Replicated Data Type (CRDT) é uma abordagem mais simples, que demonstra os mesmos conceitos fundamentais de sistemas distribuídos (sincronização de estado partilhado, consistência eventual, cache em camadas) com uma complexidade de implementação muito mais gerível.

A limitação conhecida: se duas pessoas editarem exatamente o mesmo trecho de texto dentro da mesma janela de debounce (~500 ms), uma das edições pode ser silenciosamente sobreposta pela outra. Na prática, para o volume de utilizadores simultâneos previsto por documento (o sistema limita a 20 editores em edição completa), o impacto real desta limitação é baixo, mas é uma diferença de fundo face a sistemas como o Google Docs.

10.2. Alternativa 1 - Operational Transformation (OT)

É a técnica usada historicamente pelo Google Docs. Em vez de enviar o documento completo, cada cliente envia operações discretas por exemplo, "inserir a letra 'a' na posição 42" ou "remover 3 caracteres a partir da posição 10".

O que seria necessário implementar:

- Um número de revisão por documento, incrementado a cada operação aceite pelo servidor
- Cada operação enviada pelo cliente identifica em cima de que revisão foi criada
- Uma função de transformação que, dadas duas operações concorrentes (baseadas na mesma revisão), calcula uma versão ajustada de cada uma tendo em conta o efeito da outra
- Tratamento explícito de cada combinação possível: inserção-contra-inserção, inserção-contra-remoção, remoção-contra-remoção, incluindo casos extremos (ex.: duas pessoas apagam exatamente o mesmo carácter)

A dificuldade principal do OT não é conceptual, mas de robustez: um erro subtil na função de transformação não falha de forma visível, corrompe o documento silenciosamente, muitas vezes só

detetável dias depois, numa combinação rara de edições. A própria equipa do Google Docs levou vários anos a amadurecer esta técnica em produção.

10.3. Alternativa 2 - CRDT (Conflict-free Replicated Data Type)

É a abordagem mais moderna, usada por ferramentas como o Figma ou bibliotecas como o Yjs e o Automerge. Em vez de posições absolutas no texto ("caráter 42"), cada carácter recebe um identificador único e imutável (por exemplo, {autor: "B", contador: 17}), e a sua posição é definida pela relação com os caracteres vizinhos, não por um índice. Como os identificadores nunca mudam e as operações são matematicamente comutativas, qualquer ordem de aplicação das operações converge sempre para o mesmo resultado final, sem necessidade de um servidor central a arbitrar conflitos.

O caminho realista de implementação não seria escrever um CRDT de raiz, mas sim adotar a biblioteca Yjs, que já resolve este problema e tem uma integração pronta para o Quill (y-quill). Seria necessário:

- Substituir o modelo de dados interno do Quill por um Y.Text partilhado
- Trocar a camada de transporte por um provider do Yjs , reaproveitando as salas do Socket.IO já existentes, através de um adaptador
- Alterar a persistência no PostgreSQL: o Yjs serializa o documento como um bloco binário compacto (não como HTML), o que implicaria rever o esquema da tabela de documentos e das versões

10.4. Porque não foi implementado nesta versão

A decisão de manter a técnica de snapshot + diff foi deliberada, por três razões:

- Demonstra os mesmos conceitos de fundo exigidos pela unidade curricular (sincronização de estado distribuído, consistência eventual, cache em camadas), com uma complexidade de implementação e de depuração muito menor
- O OT tem um risco elevado de bugs subtis e silenciosos, difíceis de detetar num projeto com prazo definido
- O CRDT, embora mais robusto, exigiria alterações profundas ao modelo de persistência já construído, com pouco tempo de sobra para validar exaustivamente a migração

Fica como proposta de trabalho futuro a migração para Yjs, especialmente se o sistema vier a suportar documentos com maior número de colaboradores simultâneos ou cenários de edição offline com reconciliação posterior , situações em que as garantias formais de um CRDT trazem benefícios reais.

11. Conclusão

O JI Sync demonstra, num caso de uso concreto e reconhecível, os principais desafios de um sistema distribuído: manter vários clientes em sincronia em tempo real, equilibrar desempenho e persistência através de camadas de cache, e garantir controlo de acesso consistente em múltiplos pontos de entrada do sistema.

11.1. Tecnologias utilizadas

Camada	Tecnologia
Servidor	Node.js, Express
Tempo real	Socket.IO
Base de dados	PostgreSQL
Cache	Redis
Autenticação	JWT + bcrypt
Editor de texto	Quill.js
Folha de cálculo	Handsontable
Exportação	SheetJS, jsPDF, html-docx-js, html2pdf.js